100 SQL Practice Questions

Complete Learning Path from Beginner to Expert

E-Commerce & Retail Sales Database Schema

DATABASE SCHEMA REFERENCE

- `customers` (customer_phone_id, customer_name, email, address, gst_invoice_info)
- `orders` (order_id, customer_phone [FK], order_date, store_name, total_items)
- `order_items` (order_id [FK], product_id [FK], quantity, unit_price)
- `products` (products_id, product_name, category, price)

Level 1: Beginner (Questions 1–20)

1. Retrieve all columns from the `products` table.
2. Select the `customer_name` and `email` of all customers.
3. List all products in the 'Electronics' `category`.
4. Find all products with a `price` greater than 500.
5. List all unique product categories available in the `products` table.
6. Retrieve all orders placed on '2026-01-15'.
7. List the `customer_name` and `address` of customers who have provided `gst_invoice_info`.
8. Find all orders handled by the store named 'Main Branch'.
9. List all products sorted by `price` from lowest to highest.
10. Find the top 5 most expensive products in the `products` table.
11. Retrieve all customers whose names start with the letter 'A'.
12. Select all orders where `total_items` is greater than 3.
13. List all products with a price between 100 and 500.
14. Find all customers who do not have a registered `email` address.
15. Select all order items where `quantity` purchased is 1.
16. Find all distinct store names from the `orders` table.
17. List all customers whose phone IDs end with '99'.

18. Retrieve orders placed between '2026-01-01' and '2026-01-31'.
19. Display product names and prices, renaming the price column to `unit_cost`.
20. List the 10 most recent orders sorted by `order_date` descending.

Level 2: Elementary (Questions 21–40)

21. Count the total number of customers in the `customers` table.
22. Calculate the average price of all products.
23. Find the minimum and maximum price in the `products` table.
24. Count how many orders each customer has placed (`GROUP BY customer_phone`).
25. Find the total number of items sold per store (`GROUP BY store_name`).
26. Display product categories along with the total number of products in each category.
27. Calculate the total quantity sold for each `product_id` in `order_items`.
28. Find the total number of orders placed in each month of the year.
29. Display stores that have processed more than 50 total orders (using `HAVING`).
30. List categories where the average product price is greater than 1,000.
31. Join `orders` and `customers` to show `order_id`, `customer_name`, and `order_date`.
32. Join `order_items` and `products` to display `order_id`, `product_name`, and `quantity`.
33. Find the total revenue generated per order (`SUM(quantity * unit_price)` in `order_items`).
34. Retrieve all orders placed by a specific customer using their phone ID.
35. Find the average quantity of items ordered per order item entry.
36. Display categories having more than 5 products listed.
37. Join `orders` and `order_items` to list each `order_id` with its corresponding store name and product IDs.
38. Find the total amount spent on each product across all orders.
39. List customers who have placed at least 3 orders.
40. Calculate the total units sold per store by joining `orders` and `order_items`.

Level 3: Intermediate (Questions 41–60)

41. Write a query to list `order_id`, `customer_name`, `product_name`, `quantity`, and `unit_price` using multi-table joins.
42. Find customers who have never placed an order (using `LEFT JOIN` or `NOT IN`).
43. List products that have never been ordered.

44. Calculate the total spend per customer (`customer_name`, `total_spent`).
45. Find the customer who has spent the highest total amount across all orders.
46. Categorize products as 'Budget' (price < 200), 'Mid-Range' (200–1000), or 'Premium' (> 1000) using a `CASE` statement.
47. Retrieve all orders where the calculated total amount (`SUM(quantity * unit_price)`) differs from product default prices.
48. List products whose price is higher than the average price of products in their respective category.
49. Identify stores that have sold products from every product category.
50. Find all customers who bought products from the 'Laptops' category.
51. Calculate the total sales value generated by each store.
52. Retrieve the names of customers who placed orders in both January and February 2026.
53. List the top 3 best-selling products by total quantity sold.
54. Find all orders where the total number of line items (`COUNT(product_id)`) exceeds 5.
55. Display each order along with a flag indicating whether a GST invoice was requested (`CASE` checking `gst_invoice_info`).
56. Find the average order value (AOV) across the entire platform.
57. Retrieve the details of the most recent order placed by each customer.
58. List products that were purchased with a `unit_price` lower than the official `price` in the `products` table.
59. Find the date with the highest total revenue.
60. Calculate the percentage contribution of each product category to total sales revenue.

Level 4: Advanced (Questions 61–80)

61. Rank products within each category based on price using `DENSE_RANK()`.
62. Use a CTE to calculate customer lifetime value (CLV) and select the top 10 customers.
63. Find the top-selling product in each store using `ROW_NUMBER()`.
64. Calculate the running total of sales revenue ordered by `order_date`.
65. Determine month-over-month (MoM) revenue growth using the `LAG()` window function.
66. Find the time gap (in days) between consecutive orders for each customer.
67. Identify repeat customers vs. one-time buyers using CTEs and aggregate counts.
68. Rank stores by total revenue generated per month.
69. Find orders where the total items count recorded in `orders.total_items` does not match `SUM(quantity)` in `order_items`.
70. Use a CTE to isolate orders containing more than 3 distinct product categories.

71. Calculate a 7-day moving average of daily order counts.
72. Find customers whose single order value exceeded their category-wide average spend.
73. Determine the relative percentile rank of products based on sales volume using `PERCENT_RANK()`.
74. Find products that experienced price variation (sold at different `unit_price` values over time).
75. Retrieve the first product ever purchased by each customer.
76. Write a query to pivot total sales by category across stores.
77. Use CTEs to find customers who stopped purchasing (no orders in the last 90 days).
78. Identify products frequently bought together in the same `order_id` (Market Basket Analysis).
79. Calculate the retention rate of customers month-over-month.
80. Find customers who have purchased from more than two different stores.

Level 5: Expert & Complex Analytics (Questions 81–100)

81. **Customer Cohort Analysis:** Group customers by their first purchase month and track total spend over subsequent months.
82. **RFM Analysis:** Calculate Recency (days since last order), Frequency (total orders), and Monetary value (total spend) per customer.
83. Categorize customers into RFM segments ('Champions', 'At Risk', 'Lost') based on percentile ranks.
84. **Cross-Sell Rate:** Calculate the percentage of orders containing products from more than one category.
85. **Inventory Velocity:** Determine average days passing before a product is re-ordered across the system.
86. **Price Elasticity Indicator:** Compare total quantity sold for products when sold at list price versus discounted `unit_price`.
87. Find orders containing orphan records or mismatched foreign key relationships (Data Integrity Audit).
88. Calculate Pareto principle metrics: Check if the top 20% of customers account for 80% of total revenue.
89. **Store Cross-Over:** Find customers who placed their first order at Store A but subsequent orders at Store B.
90. Identify peak ordering days/months for each store location.
91. Calculate total lost potential revenue from products that were never purchased.
92. Write an optimized query using subquery execution plans for high-volume phone ID lookup.
93. Create a query that outputs a complete JSON/structured payload per order (combining metadata and items array).
94. Calculate average basket size (`total_items` per order) trended over quarterly intervals.
95. **Churn Risk Detection:** Find active customers whose purchase frequency dropped by >50% in last 30 days compared to historical average.
96. Determine the product category that serves as the most common "gateway purchase" for new customers.

97. Calculate net revenue after excluding orders associated with missing/invalid GST numbers.
98. **Sequential Pattern Mining:** Find the most common second product category purchased after 'Smartphones'.
99. Build a complete sales summary query producing hierarchical rollups (Store -> Category -> Product) using `ROLLUP`.
100. Write a query to detect duplicate customer records based on matching emails or addresses under different `customer_phone_id`.

Solutions & SQL Code Answers

100 Practice Questions Answer Key

Level 1 Solutions

1. Retrieve all columns from products

```
SELECT * FROM products;
```

2. Select customer_name and email

```
SELECT customer_name, email FROM customers;
```

3. Products in 'Electronics' category

```
SELECT * FROM products WHERE category = 'Electronics';
```

4. Products with price > 500

```
SELECT * FROM products WHERE price > 500;
```

5. Unique product categories

```
SELECT DISTINCT category FROM products;
```

6. Orders placed on '2026-01-15'

```
SELECT * FROM orders WHERE order_date = '2026-01-15';
```

7. Customers with GST info

```
SELECT customer_name, address FROM customers WHERE gst_invoice_info IS NOT NULL;
```

8. Orders by 'Main Branch'

```
SELECT * FROM orders WHERE store_name = 'Main Branch';
```

9. Products sorted by price ascending

```
SELECT * FROM products ORDER BY price ASC;
```

10. Top 5 most expensive products

```
SELECT * FROM products ORDER BY price DESC LIMIT 5;
```

11. Customers whose names start with 'A'

```
SELECT * FROM customers WHERE customer_name LIKE 'A%';
```

12. Orders where total_items > 3

```
SELECT * FROM orders WHERE total_items > 3;
```

13. Products with price between 100 and 500

```
SELECT * FROM products WHERE price BETWEEN 100 AND 500;
```

14. Customers without an email address

```
SELECT * FROM customers WHERE email IS NULL OR email = '';
```

15. Order items with quantity = 1

```
SELECT * FROM order_items WHERE quantity = 1;
```

16. Distinct store names

```
SELECT DISTINCT store_name FROM orders;
```

17. Phone IDs ending with '99'

```
SELECT * FROM customers WHERE customer_phone_id LIKE '%99';
```

18. Orders in Jan 2026

```
SELECT * FROM orders WHERE order_date BETWEEN '2026-01-01' AND '2026-01-31';
```

19. Rename price to unit_cost

```
SELECT product_name, price AS unit_cost FROM products;
```

20. 10 most recent orders

```
SELECT * FROM orders ORDER BY order_date DESC LIMIT 10;
```

Level 2 Solutions

21. Total count of customers

```
SELECT COUNT(*) AS total_customers FROM customers;
```

22. Average product price

```
SELECT AVG(price) AS avg_price FROM products;
```

23. Min and max product price

```
SELECT MIN(price) AS min_price, MAX(price) AS max_price FROM products;
```

24. Total orders per customer

```
SELECT customer_phone, COUNT(order_id) AS total_orders
FROM orders
GROUP BY customer_phone;
```

25. Total items sold per store

```
SELECT store_name, SUM(total_items) AS total_items_sold
FROM orders
GROUP BY store_name;
```

26. Products per category

```
SELECT category, COUNT(products_id) AS product_count
FROM products
GROUP BY category;
```

27. Total quantity sold per product_id

```
SELECT product_id, SUM(quantity) AS total_quantity
FROM order_items
GROUP BY product_id;
```

28. Orders placed per month

```
SELECT EXTRACT(MONTH FROM order_date) AS order_month, COUNT(order_id) AS total_orders
FROM orders
GROUP BY order_month
ORDER BY order_month;
```

29. Stores with > 50 orders

```
SELECT store_name, COUNT(order_id) AS total_orders
FROM orders
GROUP BY store_name
HAVING COUNT(order_id) > 50;
```

30. Categories with avg price > 1000

```
SELECT category, AVG(price) AS avg_price
FROM products
GROUP BY category
HAVING AVG(price) > 1000;
```

31. Join orders and customers

```
SELECT o.order_id, c.customer_name, o.order_date
FROM orders o
JOIN customers c ON o.customer_phone = c.customer_phone_id;
```

32. Join order_items and products

```
SELECT oi.order_id, p.product_name, oi.quantity
FROM order_items oi
JOIN products p ON oi.product_id = p.products_id;
```

33. Revenue per order

```
SELECT order_id, SUM(quantity * unit_price) AS order_revenue
FROM order_items
GROUP BY order_id;
```

34. Orders by specific customer phone

```
SELECT * FROM orders WHERE customer_phone = '9876543210';
```

35. Average quantity ordered

```
SELECT AVG(quantity) AS avg_item_quantity FROM order_items;
```

36. Categories with > 5 products

```
SELECT category, COUNT(*) FROM products GROUP BY category HAVING COUNT(*) > 5;
```

37. Join orders and order_items

```
SELECT o.order_id, o.store_name, oi.product_id
FROM orders o
JOIN order_items oi ON o.order_id = oi.order_id;
```

38. Total spend on each product

```
SELECT product_id, SUM(quantity * unit_price) AS total_product_revenue
FROM order_items
GROUP BY product_id;
```

39. Customers with >= 3 orders

```
SELECT customer_phone, COUNT(order_id)
FROM orders
GROUP BY customer_phone
HAVING COUNT(order_id) >= 3;
```

40. Units sold per store

```
SELECT o.store_name, SUM(oi.quantity) AS total_units
FROM orders o
JOIN order_items oi ON o.order_id = oi.order_id
GROUP BY o.store_name;
```

Level 3 Solutions

41. Full order details join

```
SELECT o.order_id, c.customer_name, p.product_name, oi.quantity, oi.unit_price
FROM orders o
JOIN customers c ON o.customer_phone = c.customer_phone_id
JOIN order_items oi ON o.order_id = oi.order_id
JOIN products p ON oi.product_id = p.products_id;
```

42. Customers with no orders

```
SELECT c.*
FROM customers c
LEFT JOIN orders o ON c.customer_phone_id = o.customer_phone
WHERE o.order_id IS NULL;
```

43. Unordered products

```
SELECT p.*
FROM products p
LEFT JOIN order_items oi ON p.products_id = oi.product_id
WHERE oi.product_id IS NULL;
```

44. Total spend per customer

```
SELECT c.customer_name, SUM(oi.quantity * oi.unit_price) AS total_spent
FROM customers c
JOIN orders o ON c.customer_phone_id = o.customer_phone
JOIN order_items oi ON o.order_id = oi.order_id
GROUP BY c.customer_name;
```

45. Highest spending customer

```
SELECT c.customer_name, SUM(oi.quantity * oi.unit_price) AS total_spent
FROM customers c
JOIN orders o ON c.customer_phone_id = o.customer_phone
JOIN order_items oi ON o.order_id = oi.order_id
GROUP BY c.customer_name
ORDER BY total_spent DESC LIMIT 1;
```

46. Product Price Categorization

```
SELECT product_name, price,
  CASE
    WHEN price < 200 THEN 'Budget'
    WHEN price BETWEEN 200 AND 1000 THEN 'Mid-Range'
    ELSE 'Premium'
  END AS price_tier
FROM products;
```

47. Mismatched order calculated total vs list price

```
SELECT oi.order_id, SUM(oi.quantity * oi.unit_price) AS item_total, SUM(oi.quantity * p.price)
AS catalog_total
FROM order_items oi
JOIN products p ON oi.product_id = p.products_id
GROUP BY oi.order_id
HAVING SUM(oi.quantity * oi.unit_price) <> SUM(oi.quantity * p.price);
```

48. Products priced above category average

```
SELECT p1.*
FROM products p1
WHERE p1.price > (SELECT AVG(p2.price) FROM products p2 WHERE p2.category = p1.category);
```

49. Stores selling all product categories

```
SELECT o.store_name
FROM orders o
JOIN order_items oi ON o.order_id = oi.order_id
JOIN products p ON oi.product_id = p.products_id
GROUP BY o.store_name
HAVING COUNT(DISTINCT p.category) = (SELECT COUNT(DISTINCT category) FROM products);
```

50. Customers purchasing 'Laptops'

```
SELECT DISTINCT c.customer_name, c.email
FROM customers c
JOIN orders o ON c.customer_phone_id = o.customer_phone
JOIN order_items oi ON o.order_id = oi.order_id
JOIN products p ON oi.product_id = p.products_id
WHERE p.category = 'Laptops';
```

51. Total sales value by store

```
SELECT o.store_name, SUM(oi.quantity * oi.unit_price) AS store_revenue
FROM orders o
JOIN order_items oi ON o.order_id = oi.order_id
GROUP BY o.store_name;
```

52. Customers purchasing in both Jan and Feb 2026

```
SELECT customer_phone
FROM orders
WHERE order_date BETWEEN '2026-01-01' AND '2026-01-31'
INTERSECT
SELECT customer_phone
FROM orders
WHERE order_date BETWEEN '2026-02-01' AND '2026-02-28';
```

53. Top 3 best-selling products by quantity

```
SELECT p.product_name, SUM(oi.quantity) AS total_units_sold
FROM order_items oi
JOIN products p ON oi.product_id = p.products_id
GROUP BY p.product_name
ORDER BY total_units_sold DESC LIMIT 3;
```

54. Orders with > 5 line items

```
SELECT order_id, COUNT(product_id) AS line_items
FROM order_items
GROUP BY order_id
HAVING COUNT(product_id) > 5;
```

55. GST requested flag

```
SELECT o.order_id, c.customer_name,
  CASE WHEN c.gst_invoice_info IS NOT NULL THEN 'GST Requested' ELSE 'Standard' END AS
invoice_type
FROM orders o
JOIN customers c ON o.customer_phone = c.customer_phone_id;
```

56. Average Order Value (AOV)

```
WITH OrderTotals AS (
  SELECT order_id, SUM(quantity * unit_price) AS total_val
  FROM order_items GROUP BY order_id
)
SELECT AVG(total_val) AS average_order_value FROM OrderTotals;
```

57. Most recent order per customer

```
SELECT o.*
FROM orders o
INNER JOIN (
  SELECT customer_phone, MAX(order_date) AS max_date
  FROM orders GROUP BY customer_phone
) latest ON o.customer_phone = latest.customer_phone AND o.order_date = latest.max_date;
```

58. Discounted items purchase list

```
SELECT DISTINCT p.product_name, oi.unit_price AS sold_price, p.price AS catalog_price
FROM order_items oi
JOIN products p ON oi.product_id = p.products_id
WHERE oi.unit_price < p.price;
```

59. Date with highest total revenue

```
SELECT o.order_date, SUM(oi.quantity * oi.unit_price) AS daily_revenue
FROM orders o
JOIN order_items oi ON o.order_id = oi.order_id
GROUP BY o.order_date
ORDER BY daily_revenue DESC LIMIT 1;
```

60. % contribution by category to revenue

```
WITH CategorySales AS (
  SELECT p.category, SUM(oi.quantity * oi.unit_price) AS cat_revenue
  FROM order_items oi JOIN products p ON oi.product_id = p.products_id GROUP BY p.category
)
SELECT category, cat_revenue,
       ROUND((cat_revenue / (SELECT SUM(cat_revenue) FROM CategorySales)) * 100, 2) AS
pct_contribution
FROM CategorySales;
```

Level 4 Solutions

61. Rank products within category

```
SELECT product_name, category, price,
       DENSE_RANK() OVER (PARTITION BY category ORDER BY price DESC) AS price_rank
FROM products;
```

62. Top 10 Customers by CLV

```
WITH CustomerCLV AS (
  SELECT o.customer_phone, SUM(oi.quantity * oi.unit_price) AS total_clv
  FROM orders o
  JOIN order_items oi ON o.order_id = oi.order_id
  GROUP BY o.customer_phone
)
SELECT c.customer_name, clv.total_clv
FROM CustomerCLV clv
JOIN customers c ON clv.customer_phone = c.customer_phone_id
ORDER BY clv.total_clv DESC LIMIT 10;
```

63. Top-selling product per store

```
WITH StoreProductSales AS (
  SELECT o.store_name, oi.product_id, SUM(oi.quantity) AS total_qty,
         ROW_NUMBER() OVER (PARTITION BY o.store_name ORDER BY SUM(oi.quantity) DESC) as rn
  FROM orders o JOIN order_items oi ON o.order_id = oi.order_id
  GROUP BY o.store_name, oi.product_id
)
SELECT store_name, product_id, total_qty FROM StoreProductSales WHERE rn = 1;
```

64. Running total of sales revenue

```
WITH DailySales AS (
  SELECT o.order_date, SUM(oi.quantity * oi.unit_price) AS revenue
  FROM orders o JOIN order_items oi ON o.order_id = oi.order_id GROUP BY o.order_date
)
SELECT order_date, revenue,
       SUM(revenue) OVER (ORDER BY order_date) AS running_total
FROM DailySales;
```

65. Month-over-Month Revenue Growth

```
WITH MonthlyRevenue AS (
  SELECT EXTRACT(MONTH FROM o.order_date) AS mth, SUM(oi.quantity * oi.unit_price) AS rev
  FROM orders o JOIN order_items oi ON o.order_id = oi.order_id GROUP BY mth
)
SELECT mth, rev,
       LAG(rev) OVER (ORDER BY mth) AS prev_month_rev,
       ROUND(((rev - LAG(rev) OVER (ORDER BY mth)) / LAG(rev) OVER (ORDER BY mth)) * 100, 2) AS
mom_growth_pct
FROM MonthlyRevenue;
```

66. Days gap between consecutive orders

```
SELECT customer_phone, order_id, order_date,
       order_date - LAG(order_date) OVER (PARTITION BY customer_phone ORDER BY order_date) AS
days_since_prev_order
FROM orders;
```

67. Repeat vs One-time buyers

```
WITH OrderCounts AS (
  SELECT customer_phone, COUNT(order_id) AS order_cnt
  FROM orders GROUP BY customer_phone
)
SELECT
  CASE WHEN order_cnt > 1 THEN 'Repeat Customer' ELSE 'One-Time Buyer' END AS customer_type,
  COUNT(*) AS total_count
FROM OrderCounts GROUP BY customer_type;
```

68. Rank stores by monthly revenue

```
WITH StoreMonthly AS (
  SELECT o.store_name, EXTRACT(MONTH FROM o.order_date) as mth, SUM(oi.quantity * oi.unit_price)
as rev
  FROM orders o JOIN order_items oi ON o.order_id = oi.order_id GROUP BY o.store_name, mth
)
SELECT store_name, mth, rev,
       DENSE_RANK() OVER (PARTITION BY mth ORDER BY rev DESC) as store_rank
FROM StoreMonthly;
```

69. Data Discrepancy: order.total_items vs SUM(quantity)

```
SELECT o.order_id, o.total_items, SUM(oi.quantity) AS calculated_items
FROM orders o
JOIN order_items oi ON o.order_id = oi.order_id
GROUP BY o.order_id, o.total_items
HAVING o.total_items <> SUM(oi.quantity);
```

70. Orders with > 3 distinct product categories

```
WITH OrderCatCount AS (
  SELECT oi.order_id, COUNT(DISTINCT p.category) AS cat_count
  FROM order_items oi JOIN products p ON oi.product_id = p.products_id GROUP BY oi.order_id
)
SELECT order_id FROM OrderCatCount WHERE cat_count > 3;
```

71. 7-day moving average of daily orders

```
WITH DailyOrderCounts AS (
  SELECT order_date, COUNT(order_id) AS daily_orders
  FROM orders GROUP BY order_date
)
SELECT order_date, daily_orders,
       AVG(daily_orders) OVER (ORDER BY order_date ROWS BETWEEN 6 PRECEDING AND CURRENT ROW) AS
moving_avg_7d
FROM DailyOrderCounts;
```

72. Single order value > category avg spend

```
WITH CatAvg AS (
  SELECT p.category, AVG(oi.quantity * oi.unit_price) AS avg_cat_spend
  FROM order_items oi JOIN products p ON oi.product_id = p.products_id GROUP BY p.category
)
SELECT DISTINCT oi.order_id
FROM order_items oi
JOIN products p ON oi.product_id = p.products_id
JOIN CatAvg ca ON p.category = ca.category
WHERE (oi.quantity * oi.unit_price) > ca.avg_cat_spend;
```

73. Percentile rank of products by sales volume

```
WITH ProductVolume AS (
  SELECT product_id, SUM(quantity) as total_qty FROM order_items GROUP BY product_id
)
SELECT product_id, total_qty,
       PERCENT_RANK() OVER (ORDER BY total_qty ASC) as volume_percentile
FROM ProductVolume;
```

74. Products sold at varying prices

```
SELECT product_id, COUNT(DISTINCT unit_price) AS distinct_prices_count
FROM order_items
GROUP BY product_id
HAVING COUNT(DISTINCT unit_price) > 1;
```

75. First product purchased per customer

```
WITH RankedPurchases AS (
  SELECT o.customer_phone, oi.product_id, o.order_date,
         ROW_NUMBER() OVER (PARTITION BY o.customer_phone ORDER BY o.order_date ASC, o.order_id
ASC) as rn
  FROM orders o JOIN order_items oi ON o.order_id = oi.order_id
)
SELECT customer_phone, product_id FROM RankedPurchases WHERE rn = 1;
```

76. Pivot total sales by category across stores

```
SELECT o.store_name,
  SUM(CASE WHEN p.category = 'Electronics' THEN oi.quantity * oi.unit_price ELSE 0 END) AS
electronics_sales,
  SUM(CASE WHEN p.category = 'Apparel' THEN oi.quantity * oi.unit_price ELSE 0 END) AS
apparel_sales,
  SUM(CASE WHEN p.category = 'Home' THEN oi.quantity * oi.unit_price ELSE 0 END) AS home_sales
FROM orders o
JOIN order_items oi ON o.order_id = oi.order_id
JOIN products p ON oi.product_id = p.products_id
GROUP BY o.store_name;
```

77. Churned customers (no order in 90 days)

```
SELECT customer_phone_id, customer_name
FROM customers
WHERE customer_phone_id NOT IN (
  SELECT customer_phone FROM orders WHERE order_date >= CURRENT_DATE - INTERVAL '90 days'
);
```

78. Market Basket: Products frequently bought together

```
SELECT A.product_id AS product_1, B.product_id AS product_2, COUNT(*) AS times_bought_together
FROM order_items A
JOIN order_items B ON A.order_id = B.order_id AND A.product_id < B.product_id
GROUP BY A.product_id, B.product_id
ORDER BY times_bought_together DESC LIMIT 10;
```

79. Customer Retention Rate MoM

```
WITH MonthlyActive AS (
  SELECT DISTINCT customer_phone, EXTRACT(MONTH FROM order_date) as mth FROM orders
)
SELECT curr.mth, COUNT(DISTINCT curr.customer_phone) as retained_customers
FROM MonthlyActive curr
JOIN MonthlyActive prev ON curr.customer_phone = prev.customer_phone AND curr.mth = prev.mth +
1
GROUP BY curr.mth;
```

80. Customers buying from > 2 stores

```
SELECT customer_phone, COUNT(DISTINCT store_name) AS unique_stores
FROM orders
GROUP BY customer_phone
HAVING COUNT(DISTINCT store_name) > 2;
```

Level 5 Solutions

81. Cohort Analysis Query

```
WITH FirstOrder AS (
  SELECT customer_phone, DATE_TRUNC('month', MIN(order_date)) as cohort_month
  FROM orders GROUP BY customer_phone
)
SELECT f.cohort_month, DATE_TRUNC('month', o.order_date) as activity_month,
       SUM(oi.quantity * oi.unit_price) as total_revenue
FROM orders o
JOIN FirstOrder f ON o.customer_phone = f.customer_phone
JOIN order_items oi ON o.order_id = oi.order_id
GROUP BY f.cohort_month, activity_month ORDER BY cohort_month, activity_month;
```

82. RFM Base Metrics

```
SELECT o.customer_phone,
       CURRENT_DATE - MAX(o.order_date) AS recency_days,
       COUNT(DISTINCT o.order_id) AS frequency,
       SUM(oi.quantity * oi.unit_price) AS monetary
FROM orders o
JOIN order_items oi ON o.order_id = oi.order_id
GROUP BY o.customer_phone;
```

83. RFM Customer Segmentation

```
WITH RFM AS (
  SELECT o.customer_phone,
         CURRENT_DATE - MAX(o.order_date) AS recency,
         COUNT(DISTINCT o.order_id) AS frequency,
         SUM(oi.quantity * oi.unit_price) AS monetary
  FROM orders o JOIN order_items oi ON o.order_id = oi.order_id GROUP BY o.customer_phone
),
Scores AS (
  SELECT customer_phone,
         NTILE(5) OVER (ORDER BY recency DESC) as r_score,
         NTILE(5) OVER (ORDER BY frequency ASC) as f_score,
         NTILE(5) OVER (ORDER BY monetary ASC) as m_score
  FROM RFM
)
SELECT customer_phone,
  CASE
    WHEN r_score >= 4 AND f_score >= 4 THEN 'Champions'
    WHEN r_score <= 2 AND f_score >= 3 THEN 'At Risk'
    ELSE 'Lost / Hibernating'
  END AS segment
FROM Scores;
```

84. Cross-Sell Rate Percentage

```
WITH MultiCatOrders AS (
  SELECT oi.order_id, COUNT(DISTINCT p.category) as cat_cnt
  FROM order_items oi JOIN products p ON oi.product_id = p.products_id GROUP BY oi.order_id
)
SELECT
  ROUND((COUNT(CASE WHEN cat_cnt > 1 THEN 1 END) * 100.0 / COUNT(*)), 2) AS
cross_sell_order_percentage
FROM MultiCatOrders;
```

85. Average Inventory Re-order Velocity

```
WITH OrderGaps AS (
  SELECT oi.product_id, o.order_date - LAG(o.order_date) OVER (PARTITION BY oi.product_id ORDER
BY o.order_date) as gap
  FROM order_items oi JOIN orders o ON oi.order_id = o.order_id
)
SELECT product_id, AVG(gap) as avg_reorder_days_velocity
FROM OrderGaps WHERE gap IS NOT NULL GROUP BY product_id;
```

86. Price Elasticity Indicator Comparison

```
SELECT oi.product_id,
       SUM(CASE WHEN oi.unit_price < p.price THEN oi.quantity ELSE 0 END) AS
discounted_units_sold,
       SUM(CASE WHEN oi.unit_price >= p.price THEN oi.quantity ELSE 0 END) AS
full_price_units_sold
FROM order_items oi
JOIN products p ON oi.product_id = p.products_id
GROUP BY oi.product_id;
```

87. Foreign Key Audit

```
SELECT oi.order_id, oi.product_id
FROM order_items oi
LEFT JOIN orders o ON oi.order_id = o.order_id
LEFT JOIN products p ON oi.product_id = p.products_id
WHERE o.order_id IS NULL OR p.products_id IS NULL;
```

88. Pareto Analysis (Top 20% Spend Ratio)

```
WITH CustSpend AS (
  SELECT o.customer_phone, SUM(oi.quantity * oi.unit_price) as spend,
         NTILE(5) OVER (ORDER BY SUM(oi.quantity * oi.unit_price) DESC) as tile
  FROM orders o JOIN order_items oi ON o.order_id = oi.order_id GROUP BY o.customer_phone
)
SELECT
  SUM(CASE WHEN tile = 1 THEN spend ELSE 0 END) * 100.0 / SUM(spend) as
top_20_pct_revenue_share
FROM CustSpend;
```

89. Store Cross-Over Customers

```
WITH OrderedStores AS (
  SELECT customer_phone, store_name, order_date,
         ROW_NUMBER() OVER (PARTITION BY customer_phone ORDER BY order_date ASC) as rn
  FROM orders
)
SELECT DISTINCT first_s.customer_phone
FROM OrderedStores first_s
JOIN OrderedStores second_s ON first_s.customer_phone = second_s.customer_phone AND second_s.rn
> 1
WHERE first_s.rn = 1 AND first_s.store_name <> second_s.store_name;
```

90. Peak Ordering Days per Store

```
WITH DayCounts AS (
  SELECT store_name, TO_CHAR(order_date, 'Day') as day_of_week, COUNT(*) as cnt,
         ROW_NUMBER() OVER (PARTITION BY store_name ORDER BY COUNT(*) DESC) as rn
  FROM orders GROUP BY store_name, day_of_week
)
SELECT store_name, day_of_week as peak_day FROM DayCounts WHERE rn = 1;
```

91. Lost Revenue on Zero-Order Products

```
SELECT SUM(price) AS lost_potential_value
FROM products p
LEFT JOIN order_items oi ON p.products_id = oi.product_id
WHERE oi.product_id IS NULL;
```

92. Optimized History Lookup

```
SELECT o.order_id, o.order_date, oi.product_id, oi.quantity, oi.unit_price
FROM orders o
JOIN order_items oi ON o.order_id = oi.order_id
WHERE o.customer_phone = '9876543210'
ORDER BY o.order_date DESC;
```

93. Generate JSON Payload per Order

```
SELECT o.order_id,
       JSON_BUILD_OBJECT(
         'order_id', o.order_id,
         'date', o.order_date,
         'store', o.store_name,
         'items', JSON_AGG(JSON_BUILD_OBJECT('product_id', oi.product_id, 'qty', oi.quantity,
'price', oi.unit_price))
       ) as json_payload
FROM orders o
JOIN order_items oi ON o.order_id = oi.order_id
GROUP BY o.order_id, o.order_date, o.store_name;
```

94. Quarterly Average Basket Size

```
SELECT DATE_TRUNC('quarter', order_date) as quarter,
       AVG(total_items) as avg_basket_size
FROM orders
GROUP BY quarter ORDER BY quarter;
```

95. Churn Risk: 50% drop in order frequency

```
WITH HistoricalFreq AS (
  SELECT customer_phone, COUNT(order_id) * 1.0 / GREATEST(CURRENT_DATE - MIN(order_date), 1) as
hist_daily_rate
  FROM orders GROUP BY customer_phone
),
RecentFreq AS (
  SELECT customer_phone, COUNT(order_id) * 1.0 / 30 as recent_daily_rate
  FROM orders WHERE order_date >= CURRENT_DATE - INTERVAL '30 days' GROUP BY customer_phone
)
SELECT h.customer_phone
FROM HistoricalFreq h
JOIN RecentFreq r ON h.customer_phone = r.customer_phone
WHERE r.recent_daily_rate < (h.hist_daily_rate * 0.5);
```

96. Gateway Category for New Customers

```
WITH FirstOrder AS (
  SELECT customer_phone, MIN(order_id) as first_order_id FROM orders GROUP BY customer_phone
)
SELECT p.category, COUNT(*) as gateway_purchases
FROM FirstOrder fo
JOIN order_items oi ON fo.first_order_id = oi.order_id
JOIN products p ON oi.product_id = p.products_id
GROUP BY p.category ORDER BY gateway_purchases DESC LIMIT 1;
```

97. Net Revenue Excluding Invalid GST

```
SELECT SUM(oi.quantity * oi.unit_price) AS net_valid_gst_revenue
FROM orders o
JOIN customers c ON o.customer_phone = c.customer_phone_id
JOIN order_items oi ON o.order_id = oi.order_id
WHERE c.gst_invoice_info IS NOT NULL AND LENGTH(c.gst_invoice_info) = 15;
```

98. Next Product Category after 'Smartphones'

```
WITH CustomerCategorySequence AS (
  SELECT o.customer_phone, p.category,
         LEAD(p.category) OVER (PARTITION BY o.customer_phone ORDER BY o.order_date) as
next_category
  FROM orders o
  JOIN order_items oi ON o.order_id = oi.order_id
  JOIN products p ON oi.product_id = p.products_id
)
SELECT next_category, COUNT(*) as sequence_count
FROM CustomerCategorySequence
WHERE category = 'Smartphones' AND next_category IS NOT NULL
GROUP BY next_category ORDER BY sequence_count DESC LIMIT 1;
```

99. Hierarchical Rollup Query

```
SELECT o.store_name, p.category, p.product_name, SUM(oi.quantity * oi.unit_price) as revenue
FROM orders o
JOIN order_items oi ON o.order_id = oi.order_id
JOIN products p ON oi.product_id = p.products_id
GROUP BY ROLLUP(o.store_name, p.category, p.product_name);
```

100. Duplicate Customer Record Detection

```
SELECT c1.customer_phone_id as phone_1, c2.customer_phone_id as phone_2, c1.email, c1.address
FROM customers c1
JOIN customers c2 ON (c1.email = c2.email OR c1.address = c2.address) AND c1.customer_phone_id <
c2.customer_phone_id;
```